Università di Genova
DIBRIS DIPARTIMENTO
DI INFORMATICA, BIOINGEGNERIA,
ROBOTICA E INGEGNERIA DEI SISTEMI

Template for Master's Theses at DIBRIS

Matteo Dell'Amico

Master's Thesis

Università di Genova, DIBRIS
Via Dodecaneso, 35 16146 Genova, Italy
https://www.dibris.unige.it/



Computer Science MSc
Software Security and Engineering Curriculum

Template for Master's Theses at DIBRIS

Matteo Dell'Amico

Advisor: Advisor Name

Examiner: Examiner Name

August, 2026

Abstract

This is a template for the Master's thesis in Computer Science at the University of Genoa.

This document doubles as a guide for how to structure and write it. Please keep a copy of this advice and read it carefully while you write your thesis by replacing it with your material.

Table of Contents

Chapter 1 Introduction **6**

Chapter 2 Background **7**

2.1 The Application Binary Interface (ABI) 7

2.2 Foundations of Dynamic-Link Libraries 9

2.2.1 Static vs. Dynamic Linking 9

2.2.2 Memory Mapping and Address Space Isolation 9

2.3 DLLs in Binary Instrumentation 10

2.4 Paradigms of Binary Instrumentation 11

2.4.1 Execution Context and CPU State Preservation 11

2.5 Inline API Hooking Architectures 12

2.5.1 Bare-Metal Hooking: Microsoft Detours and MinHook 12

2.5.2 Cross-Platform Hooking: Frida-Gum 14

2.5.3 Passive vs. Active Hook Execution flow 17

2.5.4 Instruction Boundaries and the Role of Disassemblers 20

2.5.5 Transactional Patching 21

2.5.6 Summary of Inline Hooking Frameworks 23

2.6 Dynamic Binary Instrumentation (DBI) Architectures 24

2.6.1 Core Architectural Components 24

2.6.2 The Execution Lifecycle: The JIT Loop 25

2.6.3 The Client Interface: Instrumentation vs. Analysis Functions 27
2.6.4 High-Level Abstraction API: Intel PIN 27
2.6.5 Bare-Metal API: DynamoRIO 28
2.6.6 Summary of DBI Frameworks 29
2.7 Comparison of Approaches 30
2.8 Experimental Validation 31
2.8.1 Scenario 1: Single-Threaded Hooking and Process Injection/Ejection Mechanics 32
2.8.2 Scenario 2: Multi-Threaded Hooking and Concurrency Management 33
2.8.3 Scenario 3: Instruction-Level Assembly Tracking via DBI Engines . 34

Chapter 3 Related Work **35**
3.1 Sources 36
3.2 Style 36

Chapter 4 Methodology **38**

Chapter 5 Results **40**
5.1 Discussion 41

Chapter 6 Conclusion **42**

Chapter 1

Introduction

At its core, binary instrumentation is the technique of inserting additional code into an existing, compiled executable to observe or modify its runtime behavior without requiring access to the original source code. This capability is critical for enforcing security policies and monitoring performance.

Advisor: Devo ancora estendere questo capitolo

Chapter 2

Background

This chapter begins by introducing the foundational concepts of binary instrumentation within a Windows environment necessary to understand the proposed work. Subsequently, it analyzes state-of-the-art frameworks utilized for binary instrumentation. This overview is essential for contextualizing the compilation targets selected for our Domain-Specific Language (DSL), which are discussed in detail in Methodology (Chapter 4).

As discussed in Introduction (Chapter 1), binary instrumentation allows us to change the runtime behavior of a binary, injecting a compiled payload into an existing executable.

2.1 The Application Binary Interface (ABI)

At the foundational level of software execution, ensuring interoperability between distinct compiled modules—regardless of the specific compilers or programming languages used—requires a *standardized low-level interface*. To allow external code or instrumentation payloads to safely interact with a target process without causing unexpected runtime crashes, the execution flow must strictly adhere to the Application Binary Interface (ABI). The ABI defines the low-level rules for how functions interact at the assembly level, including how parameters are passed to a function, how return values are retrieved, and which entity is responsible for cleaning up the stack (the caller or the callee). These rules are formalized as *Calling Conventions*. The complexity of ABI compliance and the risk of runtime crashes depend heavily on the target architecture.

Windows x86 (32-bit) Architecture

In 32-bit environments, multiple calling conventions coexist, requiring developer-level alignment between the instrumentation target function and the injected payload implementation. The most common conventions include:

- `__cdecl`: Arguments are pushed onto the stack from right to left, and the *caller* cleans the stack. This is the default convention for C/C++ binaries.
- `__stdcall`: The standard convention for the Windows application programming interface (API). Arguments are pushed right to left, but the *callee* cleans the stack.
- `__fastcall`: The first two arguments are passed in the `ECX` and `EDX` registers for speed, while the rest are placed on the stack.

Understanding and matching these different conventions is critical for binary instrumentation on x86.

Windows x64 (64-bit) Architecture

To eliminate the complexity of managing multiple incompatible conventions, Windows x64 introduces a single, unified calling convention (a standard x64 ABI). In this environment, legacy keywords like `__cdecl`, `__stdcall`, and `__fastcall` are accepted by the compiler for backward compatibility but are syntactically ignored. The x64 calling convention utilizes a register-based parameter passing scheme:

- The first four integer or pointer arguments are passed from left to right in registers (`RCX`, `RDX`, `R8`, and `R9`).
- The first four floating-point arguments are passed in the `XMM0`–`XMM3` registers.
- Any arguments beyond the first four are pushed onto the stack.
- The caller is always responsible for stack cleanup.

From the perspective of binary instrumentation, the single-ABI model of x64 provides an inherent safety net: because the compiler forces both the target process and the injected payload to use the exact same assembly-level rules, calling convention mismatches are impossible. However, when writing portable instrumentation code intended to be compiled for both architectures, developers must still explicitly specify calling conventions (e.g., using macros like `WINAPI`) to prevent stack corruption when the codebase is built for x86 targets.

2.2 Foundations of Dynamic-Link Libraries

To achieve binary instrumentation in modern Windows environments, we must overcome also some operating system constraints. Since every process operates within its own isolated virtual address space, a standard process cannot directly read or write the memory of another process. Therefore, to perform binary instrumentation, we must first find a way to inject and execute our custom instrumentation payload within the context (and memory space) of the target process.

In modern computing environments, executable code can be bound to an application using one of two primary paradigms: static linking or dynamic linking.

2.2.1 Static vs. Dynamic Linking

Under the *static linking model*, any function or external resource required by an application is permanently integrated into the final binary executable (EXE) file at link time. The linker copies the precompiled object code directly from a static library file into the body of the target program. While this ensures that the application is fully self-contained, it introduces severe inefficiencies if multiple programs require the same helper functions: each separate program must contain its own physical copy of the code on disk, and each program will load a redundant copy into physical RAM upon execution.

Dynamic linking instead defers the binding of external library functions until runtime. A dynamic-link library (DLL) is an independent binary executable module that contains functions, resources, and data that can be called dynamically by other modules or applications. Rather than duplicating library code inside the host application, the operating system keeps the library code in a separate, isolated file and maps it dynamically to the calling processes as needed, drastically reducing both disk footprint and memory consumption.

2.2.2 Memory Mapping and Address Space Isolation

In modern operating systems, every running application is allocated an isolated, private virtual address space known as a process space. This architecture strictly prevents one application from directly reading, modifying, or corrupting the code or memory space of another. To enable resource sharing without violating this isolation boundaries, the operating system utilizes a technique known as virtual memory mapping. When an application loads a DLL, the operating system loader maps the linear address range of the compiled DLL binary directly into the virtual address space of the loading process. Once this mapping is completed, the functions inside the DLL behave exactly as if they were natively

compiled parts of the host application, gaining the ability to execute within its context and interact directly with its data parameters.

2.3 DLLs in Binary Instrumentation

As discussed in Section 2.2, DLLs are traditionally used to share code libraries across multiple applications, but they possess also specific characteristics that make them the standard vehicle for Windows instrumentation:

1. *Automatic Initialization:* When a DLL is successfully loaded, the Windows operating system (OS) loader automatically invokes its entry point function, `DllMain`. This allows the instrumentation framework to immediately execute its initialization sequence without requiring the host process to explicitly call any functions.

2. *Automatic Cleaning:* If the DLL is unloaded but the instrumentation code remains active in memory, subsequent executions will jump to an unmapped memory region, triggering an access violation. To ensure proper cleanup, when a DLL is successfully unloaded, the Windows OS loader automatically invokes its entry point function to execute the detachment logic.

   Code 2.1 presents the template code for an instrumentation DLL's entry points (`DllMain`).

Code 2.1: DllMain code for attaching/detaching instrumentation

```
BOOL APIENTRY DllMain (HMODULE hModule,
DWORD   ul_reason_for_call,
LPVOID  lpReserved)
{
  switch (ul_reason_for_call)
  {
    case DLL_PROCESS_ATTACH:
    // Instrumentation framework-specific logic for injecting instrumentation
    break;
    case DLL_PROCESS_DETACH:
    // Instrumentation framework-specific logic for disabling instrumentation
    break;
  }

  return TRUE;
}
```

To force a remote, unaware process to load our instrumentation DLL, a technique known as *DLL Injection* is utilized. The most common and reliable method relies both on Dynamic Linking and Windows APIs functions `CreateRemoteThread` and `LoadLibrary`. The injector process allocates memory in the target, writes the path of the DLL into that memory, and spawns a thread (using `CreateRemoteThread`) pointing to `LoadLibrary`, which reads and processes the specified DLL path.

2.4 Paradigms of Binary Instrumentation

Modern instrumentation engines generally fall into two broad architectural paradigms:

- *Inline API Hooking:* this paradigm involves modifying the original execution flow via localized jumps. It patches specific functions in memory, redirecting execution to a custom payload before returning to the original flow.
- *Dynamic Binary Instrumentation:* rather than modifying the original binary in place, dynamic binary instrumentation (DBI) engines copy, recompile, and execute instructions inside a localized sandbox known as a Code Cache.

2.4.1 Execution Context and CPU State Preservation

Building upon the constraints of the ABI Section 2.1, it is necessary to formally define what constitutes the execution context of a running process and why preserving it is paramount to process stability during any paradigm of binary instrumentation.

At any given nanosecond, the exact state of an executing thread is represented by the data held within the CPU's internal registers. On a x64 architecture, this includes General Purpose Registers, the Instruction Pointer, and various status flags (e.g., `EFLAGS`). Collectively, these values are referred to as the *CPU state*.

When an instrumentation framework successfully intercepts the execution flow, it redirects the instruction pointer to the user's custom analysis payload. Because this payload is a compiled piece of software itself, it inevitably utilizes the CPU registers to perform its own calculations, memory allocations, or logging. This introduces a critical point of failure known as *register corruption*. If the custom payload overwrites the contents of a register that the original, interrupted application was actively using (for example, to store an intermediate loop counter or a critical memory pointer), that original data is permanently lost.

When the payload finishes its execution and attempts to restore the original execution flow,

the application will blindly continue operating using the corrupted register values. Therefore, to guarantee process stability, the original CPU state must be flawlessly preserved by the instrumentation framework.

2.5 Inline API Hooking Architectures

Hooking is the specific mechanism used to instrument a function. A hook allows the redirection of the execution flow from the original code to newly injected analysis code. A standard inline hook consists of three primary logical components:

1. *The Target:* the original function whose execution we wish to intercept. The goal is to patch the first 5 to 7 bytes of the target function with an unconditional jump (`JMP`) to a user-defined payload. These initial bytes are restored to the target function once the instrumentation is complete and the framework detaches from the host process.

2. *The Detour (or Payload):* the custom function injected by the instrumentation framework. This function receives the execution flow, inspects the target function's arguments or return value, performs the desired analysis or policy enforcement, and then dictates whether the execution should continue or be blocked.

3. *The Trampoline:* in the paradigm of inline API hooking, placing an interception branch (`JMP`) inherently corrupts the target function's prologue. To maintain execution continuity, all inline instrumentation frameworks must implement an Out-of-Line Execution Bridge, universally referred to as a *"Trampoline"*. Conceptually, the trampoline is a dynamically allocated, executable memory buffer that must fulfills two architectural mandates: it securely relocates the displaced prologue instructions and unconditionally routes the instruction pointer back to the unpatched remainder of the target function.

The manner in which these frameworks handle the CPU state, the application binary interface (ABI) constraints and the hook execution flow defines their underlying architecture.

2.5.1 Bare-Metal Hooking: Microsoft Detours and MinHook

Frameworks like Microsoft Detours and MinHook are specifically designed for Windows x86/x64 and utilize a *"bare-metal" routing* approach. In this model, the hooking library has zero awareness of the CPU state or context at runtime. Consequently, they strictly enforce the ABI. The developer must possess precise knowledge of the C/C++ signatures

(argument types and return types) and calling conventions (if we need to hook in x86 architecture) of the function being hooked. Using these two frameworks requires a strict type definition of the target function pointer to avoid stack/register corruption, as illustrated in code 2.2.

Code 2.2: Detours/MinHook requires strict ABI compliance.

```
// Strict function pointer definition required by Detours
static int (WINAPI * TrueMessageBoxW)(HWND, LPCWSTR, LPCWSTR, UINT) = MessageBoxW;

// Strict function pointer definition required by MinHook
typedef int (WINAPI * MESSAGEBOXW)(HWND, LPCWSTR, LPCWSTR, UINT);
MESSAGEBOXW fpMessageBoxW = NULL;
```

Furthermore, to access the arguments correctly, the hooked version of the function must strictly adhere to the original signature, which is defined in code 2.3.

Code 2.3: Strict ABI compliance even in the Hooked Function

```
// Detours (same for MinHook)
int WINAPI HookedMessageBoxW(HWND hWnd, LPCWSTR lpText, LPCWSTR lpCaption, UINT uType
  )
```

The code structure for the hooked function is identical in both frameworks.

In code 2.4, the goal is to intercept a target process's call to `MessageBoxW` and modify the displayed dialog with custom text (`customCaption`) and altered options (`modifiedType`) during the PRE-LOGIC phase.

After this, the trampoline is called, the original function is executed with the modified arguments, and control is returned to the hooked version (POST-LOGIC).

Here, we intercept the user's choice (`result`), forcing a "yes" outcome even if the user pressed "no", and return the final result to the target process:

Code 2.4: Detours/MinHook Hooked function structure

```
// Detours (same for MinHook)
int WINAPI HookedMessageBoxW(HWND hWnd, LPCWSTR lpText, LPCWSTR lpCaption, UINT uType
  )
{
  // PRE-LOGIC
  UINT modifiedType = MB_YESNO;
  LPCWSTR customCaption = L"Hooked! Please choose:";

  // Call to the trampoline
  int result = TrueMessageBoxW(hWnd, lpText, customCaption, modifiedType);
  // POST-LOGIC
  if (result == IDNO)
```

```
    {
      std::cout << "[HOOK POST-LOGIC] Intercepted IDNO (No). Changing result to IDYES (
    Yes)!"
      << std::endl;
      result = IDYES;
    }
    else if (result == IDYES)
    {
      std::cout << "[HOOK POST-LOGIC] Intercepted IDYES (Yes). Nothing to do."
      << std::endl;
    }

    return result;
  }
```

2.5.2 Cross-Platform Hooking: Frida-Gum

Frida-Gum is the underlying low-level C library of the Frida instrumentation toolkit, representing a modern, multi-architecture approach to dynamic binary instrumentation (supporting x86/x64, ARM/ARM64, MIPS).

At the core of Frida-Gum's Interceptor engine is a centralized execution dispatcher: when an intercepted target function is invoked, execution jumps not directly to the detour function, but into a dynamically generated entry thunk, that saves the CPU state into an architecture-agnostic `GumCpuContext` structure and trigger the dispatcher, that automatically orchestrate the hook execution flow.

Unlike traditional single-paradigm frameworks like Detours or MinHook, Frida-Gum provides a flexible, dual-paradigm Interceptor API comprising *Invocation Listener Mode* and *Function Replacement Mode*

2.5.2.1 Invocation Listener Mode

Using this mode, the developer only needs to specify the `on_enter` (injected logic before the original function) and `on_leave` (injected logic after the original function), without worrying about defining strict C++ function signatures, since Frida-Gum wraps the captured `GumCpuContext` inside a high-level `GumInvocationContext` handle passed to user callbacks. In this way arguments and return values can be accessed and modified dynamically by their index, regardless of the target process architecture.

Due to these architectural differences, the implementation significantly diverges from that of Detours and MinHook. The Frida-Gum Invocation Listener implementation for inter-

cepting `MessageBoxW` is illustrated in code 2.5.

Code 2.5: Frida-Gum Invocation Listener mode (`on_enter`/`on_leave`) for `MessageBoxW`.

```
// Pre-execution callback
static void
example_listener_on_enter (GumInvocationContext * ic, gpointer user_data)
{
  // Arguments accessed by index: 0:hWnd, 1:lpText, 2:lpCaption, 3:uType
  guint originalType = GPOINTER_TO_UINT (gum_invocation_context_get_nth_argument (ic,
   3));
  guint modifiedType = MB_YESNO;
  static const wchar_t* customCaption = L"Hooked, choose:";

  gum_invocation_context_replace_nth_argument (ic, 2, (gpointer) customCaption);
  gum_invocation_context_replace_nth_argument (ic, 3, GUINT_TO_POINTER (modifiedType)
  );
}

// Post-execution callback
static void
example_listener_on_leave (GumInvocationContext * ic, gpointer user_data)
{
  int result = GPOINTER_TO_INT (gum_invocation_context_get_return_value (ic));
  if (result == IDNO)
  {
    g_print ("Intercepted IDNO. Changing result to IDYES\n");
    gum_invocation_context_replace_return_value (ic, GINT_TO_POINTER (IDYES));
  }
}
```

While Invocation Listener Mode offers architecture abstraction, it introduces an inherent limitation under specific calling conventions (such as the Windows x64 ABI): because the API for accessing arguments (`gum_invocation_context_get_nth_argument`) index exclusively into General-Purpose Registers (`RCX`, `RDX`, `R8`, `R9`), parameter extraction fails for functions receiving floating-point values, which are passed via vector registers (`XMM0`–`XMM3`).

2.5.2.2 Function Replacement Mode

To overcome the limitations of index-based parameter extraction and achieve parity with bare-metal frameworks, Frida-Gum exposes *Function Replacement Mode* via `gum_interceptor_replace` and `gum_interceptor_replace_fast`.

1. **Standard Replacement (`gum_interceptor_replace`):** Execution passes through Frida-Gum's `enter_thunk`, where `GumCpuContext` captures the CPU state and incre-

ments `trampoline_usage_counter` (identical to Invocation Listener Mode). However, instead of invoking high-level callback listeners and then jumping to the original function, the dispatcher redirects execution to a user-defined replacement function—acting as a standard C/C++ detour payload similar to Detours or MinHook. Crucially, all CPU registers (including `XMM0`–`XMM7`) are restored prior to entering the replacement function, allowing the host C/C++ compiler to handle vector parameter passing natively via standard C signatures.

2. **Fast Replacement (`gum_interceptor_replace_fast`):** Emits a direct assembly `JMP` instruction from the target prologue straight to the replacement function, bypassing Frida-Gum's dispatcher and `GumCpuContext` creation entirely, yielding an execution flow identical to bare-metal frameworks like Detours and MinHook.

The equivalent Function Replacement implementation for `MessageBoxW` is illustrated in code 2.6.

Code 2.6: Frida-Gum Function Replacement mode using native C/C++ prototypes.

```
// Function pointer type definition matching target signature
typedef int (WINAPI * TrueMessageBoxW_t)(HWND hWnd, LPCWSTR lpText, LPCWSTR lpCaption
  , UINT uType);
static TrueMessageBoxW_t TrueMessageBoxW = NULL;

// Strongly-typed C++ Detour Function
int WINAPI HookedMessageBoxW(HWND hWnd, LPCWSTR lpText, LPCWSTR lpCaption, UINT uType
  )
{
  static const wchar_t* customCaption = L"Hooked, choose:";
  UINT modifiedType = MB_YESNO;

  // Call original implementation via trampoline
  int result = TrueMessageBoxW(hWnd, lpText, customCaption, modifiedType);

  if (result == IDNO)
    result = IDYES;

  return result;
}

// Attachment via Standard Replacement Mode
gum_interceptor_replace(interceptor,
  GSIZE_TO_POINTER(gum_module_find_export_by_name(NULL, "MessageBoxW")),
  (gpointer)HookedMessageBoxW,
  NULL,
  (gpointer*)&TrueMessageBoxW);

// or Attachment via Fast Replacement Mode
```

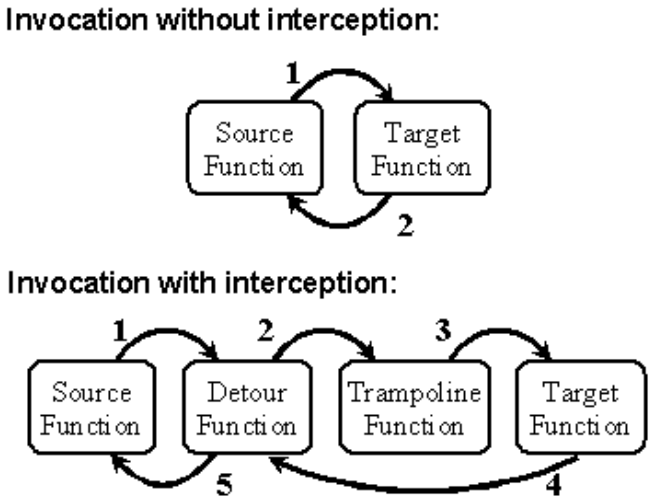
```
gum_interceptor_replace_fast(interceptor,
  GSIZE_TO_POINTER(gum_module_find_export_by_name(NULL, "MessageBoxW")),
  (gpointer)HookedMessageBoxW,
  (gpointer*)&TrueMessageBoxW);
```

2.5.3 Passive vs. Active Hook Execution Orchestration

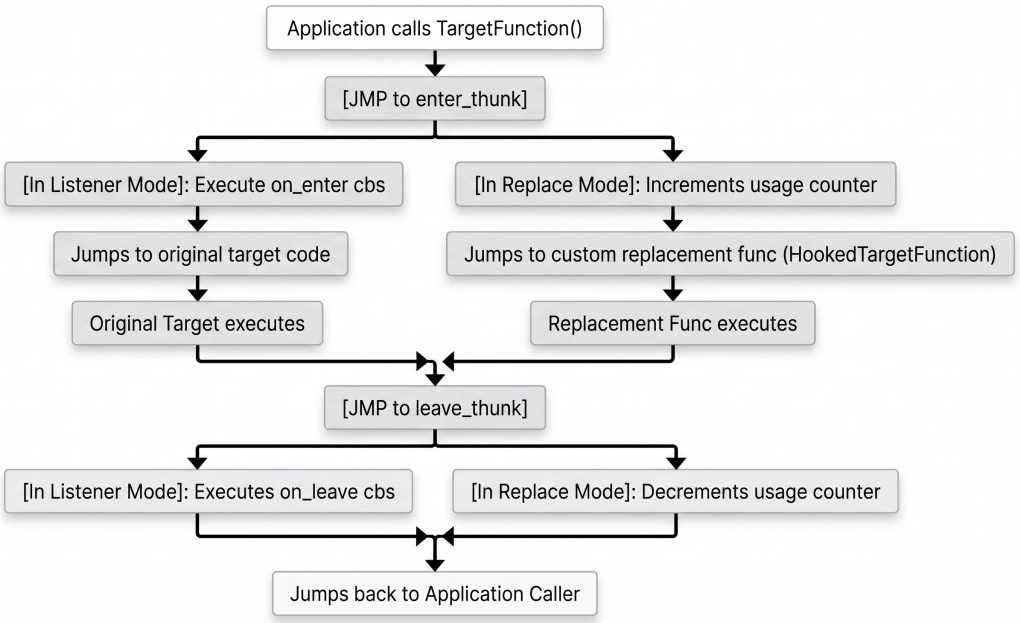
While all the presented frameworks utilize inline prologue patching, the way in which engine orchestrates Hook execution flow significantly differs:

- **Passive Orchestration (Detours, MinHook, Frida Fast Replacement):** Overwrites the target prologue with a direct assembly jump (`JMP`) to the injected payload. The engine acts purely as a passive observer, delegating all execution routing and state management to the user written detour function.
- **Active Orchestration (Frida Listener & Standard Replacement Mode):** Execution jumps into a framework-managed dispatcher. It automatically orchestrates CPU register serialization (`GumCpuContext`), and optional entry callbacks before transferring control to the target or replacement function.

To visually illustrate these operational differences, Figure 2.1 contrasts the direct, user-managed routing (Figure 2.1a) with the dispatcher-mediated execution path (Figure 2.1b).



(a) Detour/MinHook Execution Flow



(b) Frida-Gum Execution Flow

Figure 2.1: A comparison of the different Hook execution flow

2.5.3.1 Advantages of Active Orchestration

Frida-Gum employs active dispatchers in both Invocation Listener and Standard Replacement mode to deliver useful capabilities that traditional bare-metal frameworks do not automatically provide:

1. **Multi-Listener Hook:** With Detours or MinHook, developers are forced to manage fragile, nested chains of function pointers when multiple hooks target the same function. In contrast, Frida-Gum's active dispatcher maintains an internal list of subscribers (`listener_entries`) inside target related internal structure `GumFunctionContext`. The active dispatcher sequentially invokes the `on_enter` and `on_leave` callbacks of all attached listeners (e.g., logging, security monitoring, fault injection) in a single execution pass without requiring individual listeners to be aware of one another.

2. **Thread-Safe Ejection Synchronization (`gum_interceptor_flush`):** An issue in inline hooking is unmapping a payload DLL while a descheduled worker thread is executing inside hook code. The active dispatcher solves this automatically by atomically incrementing `trampoline_usage_counter` stored within the `GumFunctionContext` object upon entry (`enter_thunk`) and decrementing it upon completion (`leave_thunk`), representing the effective number of threads within the Hook logic. During detachment, calling `gum_interceptor_flush()` verifies that this counter has dropped to zero (`trampoline_usage_counter == 0`), guaranteeing that all in-flight worker threads have exited hook logic before safely restoring original bytes and unmapping memory.

3. **Automatic CPU State Preservation and Register Safety:** In bare-metal hooking, register preservation relies entirely on compiler-generated prologues matching the exact target ABI; any convention mismatch risks register corruption and crashes. In contrast, Frida-Gum's active dispatcher automatically serializes all CPU hardware registers into an architecture-agnostic `GumCpuContext` structure upon entry and restores them upon exit. Exposed to callbacks via `gum_invocation_context_get_cpu_context(ic` this mechanism guarantees process state integrity across supported architectures while allowing developers to safely inspect, log, or modify General-Purpose Registers dynamically at runtime.

2.5.3.2 Architectural Trade-offs and the Fast Replacement Bypass

Despite their capabilities, active orchestration and introduce a non-trivial performance trade-off. Execution through the framework's centralized dispatcher (`enter_thunk`) requires allocating memory for context structures and executing multiple CPU register serializations on every function invocation. For high-frequency target routines executed

millions of times per second (e.g., memory allocations or critical section acquisitions), this additional CPU overhead can degrade execution speed compared to bare-metal passive jumps.

This latency penalty explains the primary role of Frida-Gum's *Fast Replacement Mode* (`gum_interceptor_replace_fast`). By emitting a raw assembly `JMP` instruction directly from the target prologue to the replacement payload, fast replacement bypasses the `enter_thunk` dispatcher and `GumCpuContext` creation entirely. This design trades listener chaining and context serialization for bare-metal execution speed and minimal latency.

2.5.4 Instruction Boundaries and the Role of Disassemblers

Advisor: Non so se mettere questa sezione

A fundamental challenge in implementing the hook structure described above, particularly on Intel x86 and x64 architectures, is the variable-length nature of the instruction set. To detour a target function, an inline hooking framework typically needs to overwrite the first few bytes of the function's prologue with an unconditional jump (`JMP`) to the custom payload. In a standard 32-bit architecture, this jump requires 5 bytes (1 byte for the opcode, 4 bytes for the relative address). If the framework blindly overwrites exactly 5 bytes, it risks cutting an existing instruction in half. If the original prologue consists of two 3-byte instructions, overwriting 5 bytes will corrupt the second instruction, leaving a hanging, invalid byte. When the trampoline attempts to execute these copied bytes to resume the program, the CPU will encounter an invalid opcode and crash the process. To prevent this, hooking frameworks must incorporate a Length Disassembler Engine (LDE). Before applying any patches, the engine disassembles the target function's prologue to calculate exact instruction boundaries. It then copies whole, unfragmented instructions into the trampoline until it has safely relocated at least the number of bytes required for the jump (padding any remaining bytes in the original function with `NOP` instructions). The choice of disassembler heavily impacts the footprint and dependencies of the framework:

- *Microsoft Detours* relies on a proprietary, internal instruction decoder optimized specifically for standard compiler prologues.
- *MinHook* utilizes a highly optimized, custom-built Hacker Disassembler Engine (HDE32/HDE64) to calculate instruction lengths, rather than fully analyze instruction semantics.
- *Frida-Gum* requires deep semantic analysis of the instructions to achieve its context-aware routing. To accomplish this, it statically links against Capstone, a comprehensive and heavy multi-architecture disassembly framework, significantly increasing the size of the resulting DLL.

2.5.5 Transactional Patching

Another critical challenge in inline hooking is applying the memory patch (e.g., the `JMP` instruction) to a live, running process without causing a *race condition*.

In a multi-threaded application, if one thread attempts to execute a target function at the exact moment the instrumentation framework is overwriting its prologue, the CPU may attempt to execute a partially written, invalid instruction. To guarantee thread safety during the patching phase (both attaching and detaching), modern instrumentation frameworks implement a *transactional model*.

Instead of patching memory directly and immediately, the developer queues the hooks. When the transaction is finalized, the framework briefly suspends all threads in the target process (except the one applying the patch) and writes the bytes to memory atomically. Crucially, simply suspending the threads is not sufficient. A thread might have been suspended at the exact moment its Instruction Pointer (instruction pointer (IP), i.e., `EIP`/`RIP`) was pointing to the instructions inside the prologue that are about to be overwritten. To prevent this, the framework inspects the register context of each suspended thread. If a thread's IP is detected within the address range of the code being replaced, the framework dynamically relocates the IP to the corresponding instruction inside the trampoline (when attaching a hook) or back to the restored original instructions (when detaching a hook) before resuming execution.

Detours makes this transaction explicitly visible to the developer. The API requires opening a transaction, manually declaring which threads need to be stopped, attaching/detaching the specified hooks, and committing the transaction, as illustrated in code 2.7.

Code 2.7: Microsoft Detours transactional model for safe attachment and detachment.

```
// --- SAFE ATTACHMENT ---
// Start the transaction
DetourTransactionBegin();

// Enlist threads to be suspended during the patch
DetourUpdateThread(GetCurrentThread());
// here we specify possible other threads we want to stop...

// Queue the hooks
DetourAttach(&(PVOID&)TrueMessageBoxW, HookedMessageBoxW);
// here we specify possible other hooks we want to inject...

// Apply patches atomically, suspending the specified threads and flushing the
  instruction cache
DetourTransactionCommit();

// --- SAFE DETACHMENT ---
DetourTransactionBegin();
```

```
DetourUpdateThread(GetCurrentThread());
// here we specify possible other threads we want to stop...

DetourDetach(&(PVOID&)TrueMessageBoxW, HookedMessageBoxW);
// Queue removal
// here we specify possible other hooks we want to detach...

DetourTransactionCommit();
// Suspends threads, restores original bytes, resumes
```

MinHook abstracts the explicit transaction start and end calls, but enforces the exact same synchronization internally when hooks are enabled or disabled. This behavior is shown in code 2.8.

Code 2.8: MinHook internal thread synchronization during enable/disable phases.

```
// --- SAFE ATTACHMENT ---
MH_CreateHook(&MessageBoxW, &HookedMessageBoxW, reinterpret_cast<LPVOID*>(&
  fpMessageBoxW));
// here we specify possible other hooks we want to inject...

// MH_EnableHook internally suspends threads, writes the JMP, and resumes
MH_EnableHook(&MessageBoxW);
//or MH_EnableHook(MH_ALL_HOOKS); if we have more than one hook

// --- SAFE DETACHMENT ---
// MH_DisableHook internally suspends threads, restores original bytes, and resumes
MH_DisableHook(&MessageBoxW);
//or MH_DisableHook(MH_ALL_HOOKS); if we have more than one hook
```

Frida-Gum employs a similar transactional safety model through its Interceptor API, as presented in code 2.9. During attach and detach phases, atomic transaction boundaries (`gum_interceptor_begin_transaction` / `end_transaction`) batch memory patch modifications, which can be combined with explicit OS worker thread suspension (`SuspendThread` / `ResumeThread`) to eliminate patch-time race conditions.

Code 2.9: Frida-Gum transactional lifecycle for Function Replacement and Invocation Listener modes.

```
// --- SAFE ATTACHMENT ---
std::vector<HANDLE> threads = EnlistAllThreads();
for (HANDLE h : threads) SuspendThread(h);

gum_interceptor_begin_transaction(interceptor);

// Function Replacement Mode (bare-metal function replacement)
gum_interceptor_replace(interceptor, target_func, (gpointer)HookedFunc, NULL, (
  gpointer*)&TrueFunc);
```

```
gum_interceptor_replace_fast(...)

// OR Invocation Listener Mode
// gum_interceptor_attach(interceptor, target_func, listener, NULL,
  GUM_ATTACH_FLAGS_NONE);

gum_interceptor_end_transaction(interceptor);

for (HANDLE h : threads) { ResumeThread(h); CloseHandle(h); }

// --- SAFE DETACHMENT ---
threads = EnlistAllThreads();
for (HANDLE h : threads) SuspendThread(h);

gum_interceptor_begin_transaction(interceptor);

// Function Replacement Mode
gum_interceptor_revert(interceptor, target_func);

// OR Invocation Listener Mode
// gum_interceptor_detach(interceptor, listener);

gum_interceptor_end_transaction(interceptor);

for (HANDLE h : threads) { ResumeThread(h); CloseHandle(h); }
```

During the *detachment phase*, restoring the target function's original prologue bytes prevents *new* executions from entering the hook payload. However, transactional byte restoration alone is insufficient to guarantee safe DLL ejection (`FreeLibrary`). In multi-threaded applications, worker threads may already be executing inside the custom detour logic when prologue reversion occurs. If the host process unmaps the instrumentation DLL memory while a worker thread is active inside the hook logic, the thread encounters unmapped virtual memory, triggering an immediate Access Violation (`0xC0000005`).

To achieve thread-safe ejection, inline hooking architectures must supplement transactional byte restoration with a *Synchronization Protocol*. This protocol needs to tracks active worker threads executing within the hook scope and enforces a spin-wait period during detachment, ensuring all in-flight threads completely exit the hook payload before freeing resources and unmapping the binary module.

2.5.6 Summary of Inline Hooking Frameworks

To conclude the discussion on inline hooking, Table 2.1 provides a summary comparison of the three analyzed frameworks: Microsoft Detours, MinHook, and Frida-Gum.

Feature	Detours	MinHook	Frida-Gum
CPU State Preservation	Manual (Compiler)	Manual (Compiler)	Automatic (`GumCpuContext`) / Compiler
Hook Orchestration	Passive (User-controlled)	Passive (User-controlled)	Active Dispatcher (`attach`) / Passive (`replace`)
ABI Constraints	Strict (Requires C/C++ prototypes)	Strict (Requires C/C++ prototypes)	Abstracted (`attach`) / Strict (`replace`)
Disassembler	Proprietary LDE	HDE32 / HDE64	Capstone (full semantic decoder)
Architectures	x86, x64	x86, x64	x86, x64, ARM, ARM64, MIPS

Table 2.1: Comparison of the analyzed inline hooking frameworks.

2.6 Dynamic Binary Instrumentation (DBI) Architectures

For deeper execution analysis at the assembly level, DBI engines are utilized. Unlike inline hooking, which patches the original binary in place, DBI engines ensure that the target application's original code is never executed directly by the CPU. Instead, the framework employs a *Just-In-Time (JIT) compilation* mechanism to dynamically translate and instrument instruction sequences.

2.6.1 Core Architectural Components

Modern DBI frameworks rely on a unified architectural pattern comprising the following components:

1. **The Dispatcher (Control Flow Manager):** The dispatcher is the central component of the DBI engine, responsible for orchestrating the execution state. Because the application and the DBI framework share the same process but operate under different scopes, they must safely transit between "framework mode" and "application

mode". The dispatcher triggers a context-switch mechanism that carefully saves and restores the application's state whenever execution exits or enters the code cache.

2. **Just in time (JIT) Compiler:** Instead of translating the entire binary before execution, these architectures translate the application's instructions dynamically in small fragments. During this translation phase, the framework decodes and encodes (into the framework's abstraction) the original instructions and has the opportunity to insert custom instrumentation code or patches before emitting the new sequence.

3. **The Code Cache (The Execution Sandbox):** To mitigate the massive performance penalty of repeatedly translating the same code, DBI frameworks employ a software-based code cache. The translated and potentially instrumented fragments are stored in this sandboxed memory region. When the application re-executes a previously translated section of code, it runs directly from the code cache, which significantly optimizes execution speed.

4. **The Client Interface (Instrumentation API):** This is the abstraction layer exposed to the developer. It provides the necessary functions to inspect the program state, traverse instruction sequences, and specify exactly where and how instrumentation should be injected.

2.6.2 The Execution Lifecycle: The JIT Loop

To formally describe how these components interact at runtime, we can outline the execution lifecycle of a standard block of code, illustrated in Figure 2.2:

1. *Interception:* The target application attempts to execute a block of original code. The DBI engine intercepts this attempt and yields control to the Dispatcher.

2. *Cache Lookup:* The Dispatcher queries a fast-lookup hash table to check if the target memory address has already been translated. If it results in a cache hit, the CPU is redirected to the Code Cache.

3. *Translation (On Cache Miss):* If the code has not been encountered before, the JIT Compiler decodes original binary instructions one by one to form a straight-line sequence that terminates at a control transfer, such as a branch, call, or return.

4. *Instrumentation:* The engine queries the Client Interface to determine if the user wishes to instrument the block or a single instruction within it. If so, the instrumentation instructions are injected.

5. *Emission and Linking:* Most DBI frameworks use this phase to apply patches to the control-flow instructions at the end of the sequence, guaranteeing that when the block finishes running, it will redirect back to the framework rather than jumping to a native memory address. If the block ends in a direct branch to another previously translated block, the engine "links" them together directly in the cache (if it was an indirect branch, it cannot know the target before executing it), allowing the CPU to bypass the Dispatcher in future iterations. The modified block is emitted into the Code Cache.

6. *Execution:* The Dispatcher performs a context switch, restoring the application state, and commands the CPU to execute the newly cached block natively.

7. *Code Cache Exit:* Execution remains safely inside the code cache — often jumping directly from one translated block to another through "linking"—until it hits a boundary. This boundary might be an unlinked branch or an indirect branch whose target hasn't been translated yet. When this happens, the block executes an exit stub which saves the CPU's state and transfers control back to the Dispatcher, restarting the loop.

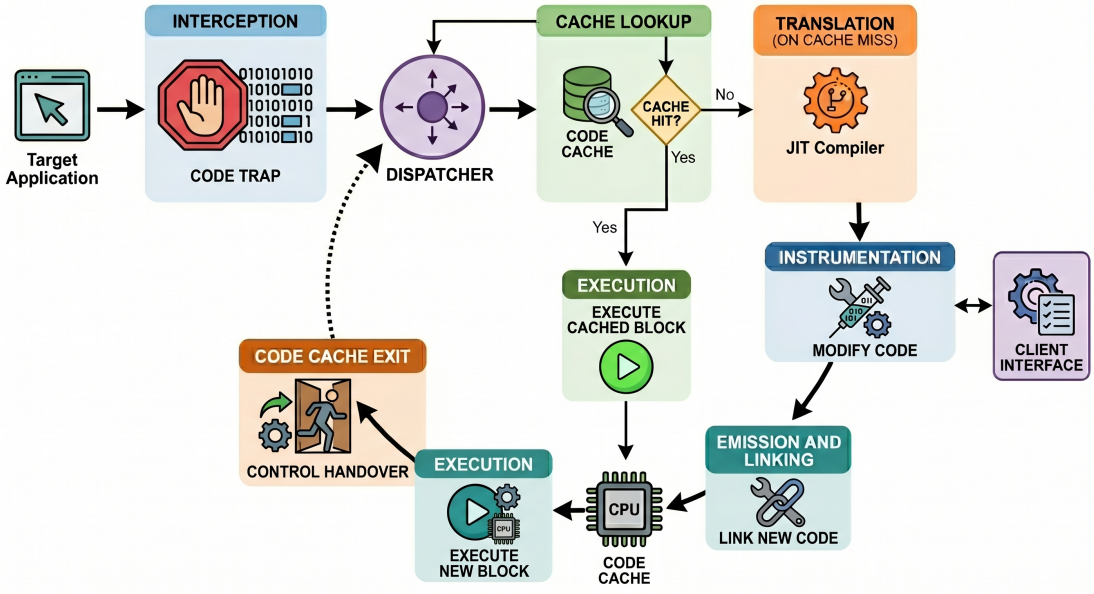


Figure 2.2: The Execution Lifecycle of a DBI framework

While DBI frameworks allow a more granular analysis of the target code, they introduce *significantly higher overhead* compared to inline hooking. Every single instruction of the program needs to be JIT-compiled before execution, even if no instrumentation is injected.

2.6.3 The Client Interface: Instrumentation vs. Analysis Functions

DBI frameworks require the developer to interact with the engine using a *two-tiered callback paradigm*. The developer must supply two distinct types of functions to the engine, each operating at fundamentally different times during the execution lifecycle:

- *The Instrumentation Function (The "Where" and "What"):* This callback is invoked by the DBI engine *only once* per newly discovered sequence of instructions during the JIT compilation phase. Its purpose is to inspect the static properties of the instructions (e.g., opcodes, operands) and decide *if* and *where* custom code should be injected. If injection is required, it uses the API to queue the insertion of the analysis function.
- *The Analysis Function (The "Action"):* This is the actual custom payload injected into the Code Cache. Unlike the instrumentation function (executed only at compile time), it is executed *every time* the CPU runs the corresponding instrumented block of code. For this reason, the analysis function must be heavily optimized and efficient.

The level of abstraction provided by the Client Interface significantly alters how developers interact with the engine.

2.6.4 High-Level Abstraction API: Intel PIN

Intel PIN is a standard academic DBI framework with a proprietary (free for non-commercial use) license. It provides a heavily abstracted C++ API.

To utilize Intel PIN, developers do not modify the core engine itself. Instead, they write a custom C++ program known as a *PINtool*. A PINtool represents the physical implementation of the Client Interface discussed previously; it is the user-defined module that contains both the Instrumentation Functions and the Analysis Functions. Architecturally, a PINtool is compiled as a standard shared library (a DLL on Windows). When the user launches a target application under PIN, the engine automatically injects the PINtool DLL into the target process's memory space. Once loaded, the engine invokes the PINtool's `main` function, allowing the tool to register its specific callbacks. In code 2.10, we can observe the granularity achievable using DBI: in this case, we can log conditional branches to determine which paths are taken during code execution.

Code 2.10: An example of a PINtool for branch logging

```
// Analysis function (Runs at execution time in the Code Cache)
```

```
void record_branch(ADDRINT instruction_address, ADDRINT target_address, BOOL taken) {

  if (taken) { /* Log Taken */ }
  else       { /* Log Not Taken */ }
}

// Instrumentation function (Runs at JIT compilation time)
void trace(INS instruction, void *v) {

  // PIN abstracts away module resolution: we want to instrument
  // only target application instructions, filter possible external libraries
  RTN rtn = INS_Rtn(instruction);
  if (RTN_Valid(rtn) && IMG_IsMainExecutable(SEC_Img(RTN_Sec(rtn)))) {
    // If it is inside the target executable, check if it's an IF branch
    if (INS_IsBranch(instruction) && INS_HasFallThrough(instruction)) {
      // Inject analysis function right before the jump happens.
      INS_InsertCall(instruction, IPOINT_BEFORE, (AFUNPTR)record_branch,
      IARG_INST_PTR,
      IARG_BRANCH_TARGET_ADDR,
      IARG_BRANCH_TAKEN,
      IARG_END);
    }
  }
}
```

2.6.5 Bare-Metal API: DynamoRIO

DynamoRIO is the open-source alternative that exposes a lower-level C API. The equivalent branch-logging example seen previously for PIN is implemented in code 2.11.

Code 2.11: DynamoRIO branch instrumentation example

```
// Analysis function
static void record_branch(app_pc instr_addr, app_pc target_addr, int taken) {

  if (taken) { /* Log Taken */ }
  else       { /* Log Not Taken */ }
}

// Instrumentation function
static dr_emit_flags_t event_bb_analysis(void *drcontext, void *tag, instrlist_t *bb,
   instr_t *inst, bool for_trace, bool translating,
void *user_data) {

  // Manual module tracking to filter out OS libraries
  app_pc instr_addr = instr_get_app_pc(inst);
  module_data_t *mod = dr_lookup_module(instr_addr);
```

```
    bool is_main_exec = false;

    if (mod != NULL) {
      module_data_t *main_mod = dr_get_main_module();
      if (main_mod != NULL) {
        if (mod->start == main_mod->start) { is_main_exec = true;
        }
        dr_free_module_data(main_mod);
      }
      dr_free_module_data(mod);
    }

    if (!is_main_exec) return DR_EMIT_DEFAULT;
    // Check if it's a conditional branch (CBR)
    if (instr_is_cbr(inst)) {
      // Inject analysis function into the basic block (bb)
      dr_insert_cbr_instrumentation(drcontext, bb, inst, (void *)record_branch);
    }
    return DR_EMIT_DEFAULT;
  }
```

Using DynamoRIO, developers can directly manipulate the instruction flow. In PIN, developers generally "wrap" existing instructions (adding a call before or after an instruction) but cannot easily modify the instruction itself. In DynamoRIO, the developer is handed a linked list of instructions (`instrlist_t`) and can easily manipulate or modify them (e.g., reordering them or moving a load instruction earlier to test for race condition).

Furthermore, because PIN is designed to be highly opaque, using `INS_InsertCall` prompts PIN's JIT compiler to automatically analyze the custom analysis function for deciding which registers need to be saved and restored based on what the C++ code actually uses. With DynamoRIO, developers can manage the state and flags manually for specialized analysis.

In conclusion, while DynamoRIO may present a steeper learning curve than Intel PIN, it affords developers greater transparency and raw power, providing fine-grained control over the translated code and the code cache.

2.6.6 Summary of DBI Frameworks

To summarize the key differences between the analyzed DBI engines, Table 2.2 compares the features and capabilities of Intel PIN and DynamoRIO.

Feature	Intel PIN	DynamoRIO
API Style	High-level C++ API	Low-level C API
Code Modification	High-level wrapping (instrumentation only before/after instructions)	Direct instruction manipulation
State/Flags Handling	Fully automatic (always analyzed and optimized by JIT compiler)	Manual or automatic (developer can choose)
Platform Support	Windows, Linux, macOS (mainly x86/x64)	Windows, Linux, macOS, Android (x86, x64, ARM, ARM64)
License	Proprietary (Free for non-commercial use)	BSD (Open Source)

Table 2.2: Comparison of the analyzed DBI frameworks.

2.7 Comparison of Approaches

To conclude the overview of the binary instrumentation ecosystem, this section provides a high-level comparison of the two paradigms discussed in this chapter: Inline Hooking and Dynamic Binary Instrumentation. Table 2.3 highlights the fundamental differences between these two methodologies.

Dimension	Inline API Hooking	Dynamic Binary Instrumentation (DBI)
Basic Mechanism	Overwrites instructions in-place (e.g., using a `JMP` in the prologue).	Translates and executes instructions in a sandboxed Code Cache via JIT compilation
Granularity	Function-level (entry and exit points).	Assembly Instruction-level.
Performance Overhead	Negligible (execution of detour and trampoline after initial jump).	Non-trivial (continuous decoding, JIT compiler loop, of all instructions).
Safety & Isolation	Lower (if not ABI compliant immediately crashes the target).	Higher (sandboxed cache; register and status flags saving is automated by the engine)
Memory Integrity	Modifies original binary pages in memory (requires altering page write permissions).	Preserves original code pages intact (modifications occur only within the cache)
Complexity	Low to Moderate (relies on basic disassembler length check and prologue displacement).	High (requires a full JIT compiler, control flow manager, and basic block linker).

Table 2.3: Comparison of the two main binary instrumentation paradigms.

2.8 Experimental Validation

To empirically validate the theoretical concepts discussed in the preceding sections and to fully understand the operational constraints, security boundaries, and low-level deployment challenges of each framework, a propedeutic suite of testing applications was developed. All source code and custom injection utilities have been organized and made available within the project's repository.[1]

The experimental activity was structured into three different scenarios, each designed to

[1] https://github.com/davde342/MasterThesis/tree/main/code/first_examples

isolate and evaluate specific architectural properties of the instrumentation engines.

2.8.1 Scenario 1: Single-Threaded Hooking and Process Injection/Ejection Mechanics

- **Test Rationale:** This preliminary scenario aimed to verify the fundamental lifecycle management of binary instrumentation within the Windows ecosystem.

  This involved implementing a custom, dual-mode utility (`injector.cpp`) to handle both the insertion and the ejection of the instrumentation payload.

  Concurrently, the test evaluated the syntactic and structural paradigms of the three inline hooking frameworks (Microsoft Detours, MinHook, and Frida-Gum) when intercepting a standard system API—`MessageBoxW` in `user32.dll`.

- **Empirical Results:** Every analyzed framework successfully intercepted the execution flow during the "Pre-Logic" phase. This capability allowed for the modification of string pointers targeting the dialog box text and caption, the reassignment of the configuration flags to `MB_YESNO`, and the runtime mutation of the return value during the "Post-Logic" phase, effectively forcing an `IDYES` outcome regardless of the user's explicit interaction.

  Furthermore, forcing the programmatic module ejection successfully triggered the `DLL_PROCESS_DETACH` branch within each hook's entry point, safely restoring the original prologue instruction bytes and confirming a flawless ejection in single-threaded environments.

- **Encountered Obstacles:** In this idealized single-threaded scenario, no big challenges occurred. Setting up Microsoft Detours was straightforward due to its thoroughly documented APIs, the abundance of reference examples, and a bare-metal paradigm that is conceptually simpler than that of Frida-Gum.

  MinHook demonstrated a similar workflow to Detours but simplified the implementation further by automatically managing transactional patching internally.

  Conversely, approaching Frida-Gum for the first time presented a steeper learning curve; given its more complex architecture, distinct paradigm, and less exhaustive documentation; comprehensively understanding its API behaviors and mapping them to my use case required a direct analysis of the huge and dense codebase within the official Frida repository.

2.8.2 Scenario 2: Multi-Threaded Hooking and Concurrency Management

- **Test Rationale:** To simulate a realistic production workload and test the instrumentation frameworks under thread concurrency, a multithreaded target engine was used (`target_multithread.cpp`). This target utilizes a synchronized task queue where a producer thread continuously queues activities with varying workloads, while three concurrent consumer threads process these tasks in parallel by invoking the exported routines `ProcessTask` and `LogTaskResult`.

- **Empirical Results:** Real-time monitoring of thread behaviors was achieved, allowing a dynamic optimization policy: the injected hook intercepted high-latency tasks (>500ms) and programmatically constrained their complexity metrics to 100ms prior to native routing, illustrating the possibility of inline hooking for runtime policy enforcement and load adaptation.

- **Encountered Obstacles:** Two major obstacles I faced in this scenario. First, performing hook detachment (`DetourDetach` in Microsoft Detours) in a concurrent context introduced a possible race condition: active worker threads could be descheduled while executing inside the hook callback, causing an immediate Access Violation (`0xC0000005`) once the library was unmapped.

  Second, in Frida-Gum, intercepting routines that process floating-point arguments (such as the `double` parameter in `LogTaskResult`) encountered an ABI mismatch because these parameters are passed via `XMM0`–`XMM3` registers under the Windows x64 ABI, while Frida expects them in other registries.

- **Implemented Mitigations:** To address the detachment race condition, an RAII-based safety synchronization primitive (`HookGuard`) was implemented using an atomic execution counter (`g_ActiveHooks`), for keeping track of threads inside the Hook logic; during `DLL_PROCESS_DETACH`, the payload enters a spin-wait loop and a grace period (`Sleep`/`SwitchToThread`) to ensure all threads exit the hook scope before module unloading.

  As detailed in Section 2.5.2, to bypass the floating-point register limitation inherent in Frida-Gum's Invocation Listener mode, the high-level callback attachment was discarded in favor of Frida-Gum's Function Replacement mode via `gum_interceptor_replace`, allowing the C/C++ compiler to handle exact register allocation natively.

2.8.3 Scenario 3: Instruction-Level Assembly Tracking via DBI Engines

- **Test Rationale:** To achieve low-level instruction granularity, a target program containing conditional branches was compiled (`PIN_dynamo_target.cpp`), alongside an Intel PIN tool (`pintool.cpp`) and a DynamoRIO client (`dynamo.c`). The rationale was to program the underlying JIT compilers to isolate all conditional branch instructions (`INS_IsBranch` / `instr_is_cbr`) and dynamically inject an analysis callback (`record_branch`) used to log the target address (taken branch).
- **Empirical Results:** Both DBI engines yielded execution trace graphs. Using IntelPIN it was also possible to write the source file code line number correspondent to the taken branch
- **Encountered Obstacles:** An initial operational challenge involved navigating the different APIs of the two frameworks; DynamoRIO exposes lower-level, bare-metal utilities compared to the highly abstracted API of Intel PIN, resulting in a steeper learning curve. Beyond API familiarity, the primary technical obstacle across both engines was attempting to log not only the target runtime address of each taken branch but also its source code line number from `PIN_dynamo_target.cpp` (compiled with debug symbols) to maximize execution trace readability.
- **Implemented Mitigations:** After several unsuccessful attempts, I have found that for solving the source line number obstacle under Intel Pin, I had to utilize the `PIN_GetSourceLocation` API. However, direct invocation of this routine inside the analysis function triggers a critical runtime error due to missing client locks. To mitigate this, calls to `PIN_GetSourceLocation` were wrapped inside a thread-safe synchronization pair using `PIN_LockClient()` and `PIN_UnlockClient()`, ensuring proper access to Pin's symbol database.

Chapter 3

Related Work

This chapter is dedicated to the work that has been done by others in the field of your thesis. Don't write a mere list of papers with a summary of each. Instead, the goal is to give a picture of how your work is situated in the context of the work of others. In this case, you should think your skeptical reader is asking "hasn't this been solved already?".

There will be two main kinds of work that you will need to discuss: similar work to yours, and work that is complementary to yours.

Similar work will try to solve the same problem you are addressing, or a similar one. Here, focus on the differences: for example, you may write something along the lines of "**DBLP:journals/x/Turing50** provides a naive definition of 'intelligent machine', which has this and that limitation. Our work is similar to that of **DBLP:journals/x/Turing50**, but we address these problems by doing this and that."[1]

Complementary work does not try to solve the same problem you are addressing, but it is important to get a full picture of your contribution. For example, if you are *not* attempting to solve a particular problem (it is "out of scope" for your work), but the solution to that problem is necessary for your work to be useful, you should explain that here.

The Related Work chapter may end up in other places of the document. Consider that it does two pieces of work at the same time: *(i)* substantiating the claim you made in the Introduction (Chapter 1) that your problem is relevant, and *(ii)* describe the context around your work to help understand its impact. If the latter part is more important, it may make sense to move this section to the end of the document, just before the Conclusion (Chapter 6). If the Background (Chapter 2) is not necessary to understand this chapter, it is also a good idea to move it just after the Introduction (Chapter 1).

[1] Do not be arrogant! I called the work of Turing naive as a joke. For sure your work will have limitations that other work may not have, and you should be honest about them.

3.1 Sources

While doing your review of related work, Google Scholar[2] is probably the easiest search engine for academic papers. You can be mindful of some bibliometric tricks to understand which papers are likely to be most influential in the field. You can see on Scholar the number of citations of a paper; CORE[3] and Scimago[4] provide rankings of conferences and journals respectively. Non-peer-reviewed articles like those published only on arXiv[5] may, of course, be important, but you should be more careful and skeptical about them. In case of doubt, ask for guidance from your advisor.

Another tool that you should *use with caution* is Semantic Scholar.[6] It provides you with large language model (LLM)-based tools to synthesize and ask questions about papers. Always double-check, because LLMs hallucinate and, as usual, you take responsibility for whatever you write in your document: treat LLMs as a useful tool you shouldn't trust.

Wikipedia is a great source of information, but you shouldn't consider it as reliable. Wikipedia articles should always provide the sources of the information they contain; you should check it yourself, judge its reliability, and cite the source.

In general, be careful about citing websites: make sure that they're authoritative, and when it's not obvious, justify why you consider them to be worth citing. Unlike this document, which I don't expect readers to print, your thesis must be readable in printed form: hence, make URLs readable in the text.

If a paper is very important for your own piece of work, have a look at the papers that cited it, to see if there are more recent works that are relevant to your thesis.

3.2 Style

Here are a few tricks about how to cite other pieces of work:

- If a paper is available in a peer-reviewed venue (journal or conference), cite that version rather than the non-reviewed one. Sometimes papers are published in two similar versions: one in a conference and an extended one in a journal. In that case, cite the journal.

[2] https://scholar.google.com
[3] https://portal.core.edu.au/conf-ranks/
[4] https://www.scimagojr.com
[5] https://arxiv.org
[6] https://www.semanticscholar.org/

- Use citations such as "The Hamiltonian cycle problem is NP-complete [**DBLP:conf/coco/Karp7** or "**DBLP:conf/coco/Karp72** showed that the Hamiltonian cycle problem is NP-complete". Do not write "[**DBLP:conf/coco/Karp72**] showed that the Hamiltonian cycle problem is NP-complete". Note, like in this case, that academics pretty much always refer to papers by citing authors' names rather than their titles.[7]
- DBLP[8] is a good source of quality Bibtex records for papers you may want to cite.
- You may have a doubt about whether to put a link in the text or use a reference in the bibliography. Use a reference if it is a document you want to refer to (even if it's not a text document, like a video or audio), use a link if you just want to refer to something but do not use it as a reference document.
- If you decide to leave links, make it so that they're usable even in printed form. The `\myhref` command of this document can be an example of how to do it.

[7] If you use LaTeX, check the source code for how to cite paper authors using the `\textcite` command.
[8] https://dblp.org

Chapter 4

Methodology

This chapter describes the core of your work: the methodology you used to solve the problem you described in the Introduction (Chapter 1). It is quite likely to have a different name (e.g., the name of the tool you wrote), and/or to be split into multiple chapters. *Methodology* suggests that you will spend space discussing your overall approach and justification for it: it is a good idea for a Master's Thesis, as opposed to just a list of things that you have done (which could be more correctly called *Method* or *Methods*, and would risk being boring and not very useful for the reader).

This chapter will likely be quite densely connected with things you have written in the Background (Chapter 2), and the results that you will show in the Results (Chapter 5). Put abundant references to help the reader navigate the document.

Try to keep a level of detail such that a skilled reader will be able to reproduce your result, but do not include unnecessary details that the reader can find out for themselves (e.g., the list of commands to perform a standard task). That is indeed valuable information that can help reproducibility, but if you do not think it is needed to understand your work, its place is likely to be in an appendix or in the documentation of the code you wrote, if you decide to release it.

This chapter should include work that has been done by you. If that is not the case, consider moving it to the Background (Chapter 2), or make it very clear that you are not the author. The same thing applies to images: if they aren't yours, you should cite the source.

The results of your work should generally not be included in this chapter. In some cases, preliminary results can be included to explain why you took a certain direction in your work (e.g., you took a choice rather than another because you measured that it was more efficient), but the overall evaluation of your work should be in the Results (Chapter 5).

You are likely to include algorithms in pseudocode. Check relevant LaTeX packages[1] for that.

[1] https://www.overleaf.com/learn/latex/Algorithms

Chapter 5

Results

Here you will present the results of your work. Your skeptical reader is now asking themself "is this working?". You will show here to what extent it does. Be honest about the limitations: if your system doesn't work in some cases, you should say so (and explain where and why, if you can). Like the Methodology chapter, this one may end up being split into multiple chapters.

Keep in mind that this shouldn't be a mere dump of experimental results; you're rather *teaching* your reader how and to what extent you managed to solve the problem you described in the Introduction (Chapter 1). If you have additional results that may be useful but are not necessary to understand the points you're making (e.g., you evaluated your system on multiple datasets and the results all tell the same story), the place for them is in an appendix.

This chapter should have a lot of links to the methodology chapter(s), because you're evaluating the choices you made there. If you developed a system made of multiple parts, make sure that you test them separately and together, so that the reader can understand how important each part is.

This chapter is likely to be quite full of figures and tables. Try to make them as informative as possible (e.g., use multiple lines in the same plot if possible). Showing graphs effectively is a complex art; try to spend some time on it and ask for guidance from your advisor. Whenever you have a figure or a table, make sure that you refer to it in the text (after all, if it's not referred to in the text it means it has no part in the story you're telling, so it has no place in this Chapter). Always use the text (and the caption, if you can) to explain what you want the reader to understand from the figure.

If you are using LaTeX, it will automatically place figures, tables, algorithms, etc. somewhere in the text. These parts of the documents are called *floats* because their position

Column 1	Column 2	Column 3
aaa	2	3.42
bbb	505	6.00
ccc	8	901.02

Table 5.1: A table using the LaTeX `booktabs` package. Note there are no vertical rules. In case you want to do more fancy stuff (e.g., merging cells), check also the `multirow` and `multicol` packages. It's generally a good idea for readability to align text to the left and numbers to the right (use the same number of significant digits).

floats around depending on LaTeX's will. There are options to advise LaTeX about where floats should be placed. My advice is not to waste too much time on this and trust that they will end up in a good place. Generally, write the floats' code before you refer to them: they will never be placed before the place they appear in.

For Figures, generally use vectorial formats (e.g., PDF, SVG) where it makes sense if you can. They will result in higher-quality images that are in most cases also smaller, leading to shorter compiling times and a smaller resulting PDF.

Tables look better without vertical rules: an example is in Table 5.1.

5.1 Discussion

At the end of this chapter, take a step back and summarize all the results so that the reader can understand the big picture. Sometimes, this becomes large and important enough to be worth a chapter on its own.

Chapter 6

Conclusion

You are finally done! Here you will summarize for the reader what you have taught them through the document, and the main takeaways of your work you would like them to remember.

It is also a good idea to discuss the limitations of your work and your views about what can be possible future work.

Bibliography

[BGA03] Derek Bruening, Timothy Garnett, and Saman P. Amarasinghe. "An Infrastructure for Adaptive Dynamic Optimization". In: *1st International Symposium on Code Generation and Optimization (CGO 2003), 23-26 March 2003, San Francisco, CA, USA*. IEEE Computer Society, 2003, pp. 265–275. DOI: 10.1109/CGO.2003.1191551. URL: https://doi.org/10.1109/CGO.2003.1191551.

[Fri23] Frida Developers. *Frida Documentation: C API (Frida-Gum)*. 2023. URL: https://frida.re/docs/c-api/.

[Fri26] Frida Developers. *Frida-Gum: Cross-platform instrumentation and introspection library*. 2026. URL: https://github.com/frida/frida-gum.

[HB99] Galen C. Hunt and Doug Brubacher. "Detours: Binary Interception of Win32 Functions". In: *Proceedings of the 3rd USENIX Windows NT Symposium, Seattle, WA, USA, July 12-13, 1999*. USENIX Association, 1999, pp. 135–143. URL: https://www.cs.columbia.edu/~junfeng/10fa-e6998/papers/detours.pdf.

[Int26] Intel Corporation. *Pin - A Dynamic Binary Instrumentation Tool*. 2026. URL: https://software.intel.com/sites/landingpage/pintool/docs/99776/Pin/doc/html/index.html.

[Kag26] Tsuda Kageyu. *MinHook: The Minimalistic x86/x64 API Hooking Library for Windows*. 2026. URL: https://github.com/TsudaKageyu/minhook.

[Luk+05] Chi-Keung Luk, Robert Cohn, Robert Muth, Harish Patil, Artur Klauser, Geoff Lowney, Steven Wallace, Vijay Janapa Reddi, and Kim Hazelwood. "Pin: building customized program analysis tools with dynamic instrumentation". In: *Proceedings of the 2005 ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI 2005, Chicago, IL, USA, June 12-15, 2005*. Ed. by Vivek Sarkar and Mary W. Hall. ACM, 2005, pp. 190–200. DOI: 10.1145/1064978.1065034. URL: https://doi.org/10.1145/1064978.1065034.

[Mic26] Microsoft Corporation. *Detours: Software package*. 2026. URL: https://github.com/microsoft/Detours.

[Sar96] Debabrata Sarma. *DLLs for Beginners*. Technical Article. Microsoft Developer Support, Nov. 1996. URL: http://biblioteca.ucf.edu.cu/biblioteca/libros_digitales/informatica/programacion/c-1/Debabrata%20Sarma-DLLs%20for%20Beginners.pdf.

[The26] The MITRE Corporation. *Process Injection: Dynamic-link Library Injection, Sub-technique T1055.001*. 2026. URL: https://attack.mitre.org/techniques/T1055/001/.